PennCloud

Distributed Cloud Platform

Complete System Design Document

Architecture - Replication - Fault Tolerance - Storage - Tradeoffs

---

CIS 5550 - Penn Engineering - Team 05

Table of Contents

1. System Overview 3

2. Three-Tier Architecture 4

3. Key-Value Store Design 6

4. Replication & Fault Tolerance 8

5. Coordinator Design 10

6. Frontend Server Design 11

7. Drive - Virtual Filesystem 13

8. Storage Quota 15

9. Email System 17

10. Bottlenecks & Performance 18

11. Design Tradeoffs 20

12. KV Wire Protocol 22

13. Startup & Configuration 23

14. Security Considerations 24

15. Summary 25

1. System Overview

PennCloud is a distributed cloud platform built entirely in C++17 with no external frameworks. It delivers four user-facing services - Email, File Drive, Live Chat, and an Admin Console - on top of a custom replicated key-value store. Every component was written from scratch: HTTP server, SMTP server, KV tablet storage, primary-backup replication, coordinator-driven failure detection, and a session management layer.

1.1 Service Catalogue

Email:	Send/receive MIME email. Supports attachments, folder management (inbox / sent / trash), and real-time SSE push for new-mail notification.
File Drive:	Hierarchical virtual filesystem stored in KV. Upload files up to 64 MB, create folders, move/copy/delete, and track per-user storage quota.
Live Chat:	Real-time group chat channels backed by KV rows; SSE streaming for instant message delivery.
Admin Console:	Kill / restart / redirect any of the three frontend nodes from the browser. Monitor node liveness. Coordinator-driven failover visible in real time.

1.2 Technology Choices

The entire system is written in C++17 with POSIX sockets (no Boost.Asio, no libuv, no Nginx, no external HTTP libraries). This was a deliberate constraint from the course specification and forced careful hand-crafted protocol design. The result is a system with very low external dependencies - the only third-party code is fpdf2 (this document generator) and the standard library.

C++17:	Course requirement; gives direct control over threading, memory, and socket I/O with no GC pauses.
POSIX TCP sockets:	Maximum portability; works inside Docker containers that lack lsof/fuser/ss.
std::shared_mutex (rows):	Allow many concurrent reads on hot rows while exclusive writes are rare.
std::mutex (WAL):	WAL writes must be strictly serialised to maintain LSN monotonicity.
Bloom filter per tablet:	O(1) probabilistic guard to skip disk I/O for non-existent rows.
SSE over WebSocket:	Server-Sent Events require only standard HTTP; no upgrade negotiation, easier to implement from scratch.

2. Three-Tier Architecture

PennCloud uses a strict three-tier architecture. No tier bypasses any other tier - browsers talk only to frontends, frontends talk only to KV nodes (via coordinator for routing), and KV nodes replicate among themselves.

2.1 Tier Map

```
Browser (HTTP/1.1)
     |
+-----------+    +-----------+    +-----------+
|  fe1:8080  |   |  fe2:8090  |   |  fe3:8095  |   <- Frontend tier
+-----------+    +-----------+    +-----------+
     |                |                |
     +--------+-------+--------+-------+
              | KV client (TCP)
+-----------+    +-----------+    +-----------+
| node1:5000 |   | node2:5001 |   | node3:5002 |   <- KV / Tablet tier
+-----------+    +-----------+    +-----------+
     |                |                |
     +--------+-------+--------+-------+
              | Heartbeat / Election
         +------------+
         | Coordinator|  :6000            <- Coordinator
         +------------+
              |
       +---------------+
       | smtp_server   |  :2525             <- SMTP tier
       +---------------+
```

2.2 Frontend Tier

Three frontend processes (fe1, fe2, fe3) run on ports 8080, 8090, and 8095. Each maintains a pool of up to 32 worker threads handling HTTP/1.1 connections. Frontends are fully stateless with respect to user data - all state lives in the KV store. Session cookies map to KV-backed session objects, so any frontend can serve any session.

- fe1: primary frontend, handles the bulk of traffic under normal operation.
- fe2 / fe3: standby frontends; promoted by admin action or failover.
- Redirect stubs: a killed frontend is replaced by a lightweight feserver process that returns HTTP 302 to a live peer for all page requests and HTTP 503 for all /api/* requests (so the health probe can distinguish stubs from live servers).

2.3 KV / Tablet Tier

Three KV node processes (node1, node2, node3) host three tablets. Each tablet covers a letter-range of the KV key space and is replicated across all three nodes.

```
tabletA  ->  rows whose first character is in [a-g]
tabletB  ->  rows whose first character is in [h-p]
tabletC  ->  rows whose first character is in [q-z, 0-9, special]
```

Each tablet has one primary and up to two backups at any point in time. The primary handles all writes;

backups receive replicated log entries. Reads can be served from any replica (the frontend always routes to the coordinator-designated primary for writes and tries the primary first for reads).

2.4 Coordinator

A single coordinator process runs on port 6000. It does not store user data. Its only job is to track which KV nodes are alive and which replica is primary for each tablet. It drives heartbeats and primary election.

2.5 SMTP Server

A dedicated SMTP server listens on port 2525 (mapped to 25 externally). It accepts inbound mail for @penncloud.com and @penncloud domains, validates the recipient exists in the KV store, and writes the message directly into the KV mailbox. Outbound mail for external domains is forwarded via MX lookup or a configured relay.

3. Key-Value Store Design

The KV store is a custom bigtable-inspired store: rows are string keys, each row contains a map of column-name -> value (both arbitrary byte strings). The API has four operations: GET, PUT, CPUT (compare-and-put / atomic swap), and DELETE.

3.1 Data Model

```
GET    row col            -> value  (or NOT_FOUND)
PUT    row col value      -> OK
CPUT   row col old new    -> OK | FAIL (old value mismatch)
DELETE row col            -> OK
```

CPUT is the key primitive enabling safe concurrent updates. The frontend uses it extensively for directory entry appending, mail index updates, and session management - anywhere two concurrent writers could corrupt a shared value.

3.2 In-Memory Layout

Each tablet keeps all data in an in-memory hash map:

```
std::unordered_map<string, unique_ptr<RowData>>  rows_
struct RowData {
    std::shared_mutex mu;
    std::unordered_map<string, string> cols;
};
```

Two-level locking: rows_ is protected by a shared_mutex allowing concurrent readers. Each RowData has its own shared_mutex for column-level concurrency. This means two threads writing to different rows never block each other.

Why unordered_map?

O(1) average-case lookup. The KV store is a pure lookup table - there is no range-scan API - so hash-map wins over B-tree here. The bloom filter provides a fast probabilistic pre-check before any map lookup.

3.3 Bloom Filter

Each tablet maintains a per-tablet Bloom filter seeded with all known row keys. Before any GET, CPUT, or DELETE touches rows_, the bloom filter is checked. If the filter says 'definitely not present', the operation returns immediately without taking any lock. False positives are harmless (they fall through to the real check); false negatives cannot happen - the filter is add-only.

- Benefit: eliminates lock contention for non-existent row lookups (e.g., checking quota on a new user).
- Trade-off: small memory overhead per tablet (~8 KB for a 64-bit Bloom filter with a 1% false-positive rate at 10k rows).

3.4 Write-Ahead Log (WAL)

Every PUT and DELETE is written to the WAL before touching in-memory rows. This ensures crash recovery: on restart, the tablet replays the WAL to restore any operations that happened after the last checkpoint.

WAL Binary Format (per entry)

```
[4 bytes]  magic = 0xDEADBEEF  (little-endian)
[1 byte ]  op type: 0x01 = PUT, 0x02 = DELETE
```

```
[4 bytes]  row length
[4 bytes]  column length
[4 bytes]  value length  (0 for DELETE)
[N bytes]  row key
[M bytes]  column key
[V bytes]  value  (absent for DELETE)
```

The magic header lets the replay loop detect truncated or corrupted entries and stop safely rather than applying garbage data.

WAL Locking

All WAL writes are serialised by a dedicated wal_mu_ mutex (std::mutex). This is separate from the row-level shared_mutex so WAL I/O does not block concurrent readers. The WAL mutex also serialises LSN assignment - the LSN is incremented atomically inside the WAL lock, guaranteeing that LSN order matches WAL entry order.

3.5 Checkpointing

Periodically (triggered by the coordinator or on snapshot-sync), the tablet writes a checkpoint: a binary snapshot of the entire in-memory state, then truncates the WAL. This bounds recovery time to only replaying the WAL entries since the last checkpoint.

Checkpoint Binary Format

```
[4 bytes]  CKPT_MAGIC
[8 bytes]  checkpoint_version  (monotone counter)
[8 bytes]  LSN at checkpoint time
[4 bytes]  number of rows
for each row:
    [4 bytes]  row key length
    [N bytes]  row key
    [4 bytes]  number of columns
    for each column:
        [4 bytes]  col key length
        [M bytes]  col key
        [4 bytes]  value length
        [V bytes]  value
[bloom filter serialization]
```

The checkpoint is written atomically: first to a .tmp file, then renamed over the live .ckpt file (rename is atomic on Linux/macOS). This prevents a partial checkpoint from corrupting recovery.

Why checkpoint_version?

When a stale replica reconnects, the primary sends either a WAL delta (if the replica's checkpoint version matches) or a full snapshot (if not). The checkpoint_version allows the primary to detect whether the replica's WAL base is compatible, avoiding replaying deltas that no longer correspond to the WAL.

4. Replication & Fault Tolerance

4.1 Primary-Backup Model

PennCloud uses synchronous primary-backup replication (also called primary-secondary or primary-replica). There is exactly one primary per tablet at any time. All writes go to the primary; backups apply writes in the same order via forwarded WAL entries.

- Writes: primary writes WAL + memory, then forwards to all configured replicas.
- Reads: always from the primary (no stale reads). Frontends discover the primary via the coordinator.
- Quorum: a write succeeds after being acknowledged by a strict majority (floor(N/2)+1 nodes). With N=3, quorum=2 (the primary + 1 replica).

4.2 LSN (Log Sequence Number)

Every write is assigned a monotonically increasing uint64_t LSN. The primary assigns the LSN inside the WAL lock, ensuring strict ordering. Each replica tracks the highest LSN it has applied. The coordinator uses LSN information to pick the best-LSN replica as the new primary during election.

```
// Inside wal_mu_ lock:
uint64_t new_lsn = lsn_.fetch_add(1) + 1;
// new_lsn is unique and strictly greater than all prior LSNs
```

LSNs are embedded in every REPLICATE message. The backup checks that the incoming LSN is exactly current+1 - any gap triggers a resync request.

4.3 Replication Protocol

Normal path (write)

```
Frontend  ->  Primary (TCP):  PUT row col value
Primary:
  1. acquire primary_write_mu_
  2. write WAL entry (assigns LSN N)
  3. update in-memory rows_
  4. forward "REPLICATE tabletX N PUT ..."
     to all configured replicas
  5. wait for quorum ACKs  (with 500ms timeout)
  6. release primary_write_mu_
  7. reply +OK LSN=N to frontend
Each backup:
  1. verify expected_lsn == local_lsn + 1
  2. write WAL entry
  3. update in-memory rows_
  4. reply +OK LSN=N
```

Gap detection and resync

If a backup receives LSN N but its local LSN is < N-1, it replies with an error and the primary marks that replica as needing resync. On the next coordinator heartbeat, the replica is told to sync from the primary.

Sync protocol (DELTA vs SNAPSHOT)

When a replica reconnects or falls behind, it sends: SYNC_FROM tabletX <ckpt_ver> <lsn>

- If ckpt_ver matches and lsn is within the current WAL range: primary sends a WAL delta (only the

missing entries). The replica applies the delta in-order.

- Otherwise: primary sends a full snapshot blob (entire in-memory state + LSN + ckpt_ver). The replica loads the snapshot and checkpoints it to disk.

```
SYNC_FROM tabletA 3 1042\r\n
<- +DELTA 4096\r\n<4096 bytes of WAL entries>
   or
<- +SNAPSHOT 102400\r\n<102400 bytes of full state blob>
```

4.4 Fault Tolerance Scenarios

Scenario 1: One replica crashes

System continues with quorum of 2/3. Writes still succeed. The dead replica is detected by the coordinator within ~1.5 seconds (3 missed heartbeats at 500ms intervals). When it restarts, it sends SYNC_FROM and the primary sends a delta or snapshot to bring it up to date.

Scenario 2: Primary crashes

Coordinator detects the failed primary after ~1.5 seconds. It selects the replica with the highest LSN as the new primary and broadcasts the new primary assignment to all nodes and frontends. The new primary begins accepting writes immediately. If the previously dead primary restarts, it is demoted to secondary and syncs from the new primary.

Scenario 3: Two replicas crash (minority left)

Writes fail quorum check (only 1/3 ack). The system enters a degraded mode where the primary logs locally but cannot achieve quorum. It logs a warning and returns success to the frontend in degraded mode (prioritising availability over strict consistency). This is a deliberate trade-off for a cloud demo system - in production, writes would be rejected.

Scenario 4: Frontend crash

Frontend state is stateless - no user data is lost. The admin console detects the crashed frontend (via the HTTP /api/admin/status probe) and can launch a redirect stub or restart the server. User sessions persisted in KV are still valid on any other frontend.

Scenario 5: Coordinator crash

The KV nodes continue serving reads and writes using the last known primary assignment. No new primary elections can happen until the coordinator restarts. This is acceptable because elections only happen on node failure, which is rare.

4.5 Primary Election Algorithm

When the coordinator detects a failed primary (3 consecutive missed heartbeats), it runs the following election:

- Collect LSNs from all alive replicas for the failed tablet.
- Pick the replica with the highest LSN (most up-to-date).
- Send PROMOTE message to the winning replica.
- Send ADD_REPLICA messages to all other alive replicas pointing at the new primary.
- Broadcast the new topology to all frontends.

Why highest LSN? Because it minimises data loss. The highest-LSN replica has applied the most writes, so promoting it means the fewest writes need to be re-applied or lost.

5. Coordinator Design

The coordinator is the central brain for cluster topology. It does not store user data and is not in the critical path of reads or writes - frontends cache the primary address and contact KV nodes directly.

5.1 Heartbeat Loop

The coordinator runs a background heartbeat loop every 500ms. It pings each registered KV node with a PING command over TCP and expects a PONG response within the timeout. If a node misses 3 consecutive heartbeats (~1.5 seconds), it is declared dead.

```
Coordinator  ->  KV Node:  PING\r\n
KV Node  ->  Coordinator:  +PONG\r\n
```

Lock design in heartbeat loop

The heartbeat loop snapshots the node list under a shared (read) lock, releases the lock, performs all TCP pings outside the lock (avoiding holding a lock during network I/O), then re-acquires an exclusive (write) lock only to write back the results. This prevents the coordinator from becoming a bottleneck when many nodes are registered.

5.2 Node Registration

KV nodes register themselves with the coordinator at startup by sending a REGISTER message containing their node ID, host, port, and tablet name. The coordinator records this and begins including the node in heartbeat cycles.

```
KV Node  ->  Coordinator:  REGISTER node1 127.0.0.1 5000 tabletA\r\n
Coordinator  ->  KV Node:   +OK\r\n
```

5.3 Frontend Topology Updates

When a primary changes (election or manual promotion), the coordinator broadcasts a topology update to all connected frontends. Frontends update their KV client's primary routing table. Between broadcasts, frontends use their cached routing information.

5.4 Line Length

The coordinator uses a custom line reader with a maximum buffer of 8192 bytes. This is necessary because SYNC_FROM messages can carry large metadata. The 8192 limit prevents unbounded memory growth from malformed clients.

6. Frontend Server Design

6.1 HTTP Server

Each frontend runs a custom HTTP/1.1 server on a dedicated port. It accepts connections in a loop and dispatches each connection to a worker thread from a fixed-size thread pool (default 32 threads). The HTTP parser handles: method, path, query string, headers, and body. It supports chunked transfer encoding for large file uploads.

- Thread pool size: configurable via --threads flag (default 32).
- HTTP methods: GET, POST, PUT, DELETE.
- Content-Length body reads: supports up to 64 MB per request.
- Static file serving from configurable --static directory.

6.2 Route Dispatch

Routes are matched by a simple prefix/pattern matcher. Parameterised routes (:uid, :folder) extract values into a params map. Route handlers are registered as function pointers or lambdas. The dispatch table is built at startup; no dynamic registration occurs after the server starts.

6.3 KV Client

Each frontend process maintains a KVClient for each KV node. The client holds a ConnectionPool with 2 persistent TCP connections per node (reused across requests). Socket timeout is 350ms; the client retries up to 4 times with exponential backoff (0/75/150/250ms) before failing.

Why 2 connections per node?

One connection is rarely sufficient because the HTTP thread pool has 32 threads, all potentially making KV calls simultaneously. Two persistent connections reduce connection setup overhead and allow some parallelism without the complexity of a full connection pool. The trade-off: a global mutex per client serialises access to the connection pool, making it a bottleneck under high concurrency (see Section 10).

Retry backoff

```
attempt 0:  no wait  (immediate retry)
attempt 1:  75ms wait
attempt 2:  150ms wait
attempt 3:  250ms wait
-> give up, return error to HTTP handler
```

6.4 Session Management

Sessions are backed by the KV store under the key session:<SID>. The session ID is a 128-bit random value from /dev/urandom, hex-encoded to 32 characters. Sessions expire after 24 hours (stored as a Unix timestamp in the session value). On every authenticated request, the frontend reads the session from KV and validates the expiry.

- SID is sent as an HTTP cookie (Set-Cookie: sid=<SID>; HttpOnly; Path=/).
- Cookie is HttpOnly to prevent JavaScript access (XSS mitigation).
- Session destroy is a KV DELETE of the session row.
- Because sessions are in KV, any frontend can serve any session - no sticky sessions needed.

6.5 High Availability (Admin Kill/Restart/Redirect)

The admin console can kill, restart, or redirect-stub any frontend node. When a frontend is killed, it launches a replacement redirect stub before dying.

Self-kill sequence

The challenge: a process cannot kill itself and then launch a replacement - the replacement launch would die with the process. Solution: a shell background subshell is launched before the process kills itself. The subshell is a grandchild of the server process and survives SIGKILL.

```
// Server code (runs in HTTP handler, then returns response):
std::string cmd = "("
    "sleep 0.15; "  // let HTTP response escape
    "ps -axo pid=,command= | awk '/--port 8080/{print $1}' | xargs kill -KILL; "
    "sleep 0.05; "
    "nohup ./feserver --port 8080 --redirect-to http://127.0.0.1:8090 &"
") >/dev/null 2>&1 &";
::system(cmd.c_str());
```

Redirect stub

The stub is a feserver process launched with --redirect-to http://peer:port. It returns HTTP 302 for all page requests and HTTP 503 for all /api/* requests. The health probe uses HTTP 200 vs 503 to distinguish a live server from a stub.

6.6 Server-Sent Events (SSE)

SSE is used for real-time inbox notifications and chat. An SSE connection is a long-lived HTTP response with Content-Type: text/event-stream. The server polls KV every 500ms and pushes a data: event when new messages arrive.

- SSE slots are limited to threads/4 (default 8) to prevent SSE connections from consuming all thread pool slots and starving normal requests.
- Slot acquisition uses a CAS loop on an atomic<int> counter.
- Connection drop is detected by write() failure to the client socket.

7. Drive - Virtual Filesystem

7.1 KV Schema

The virtual filesystem is entirely encoded in KV rows. There is no separate filesystem layer - everything is key-value lookups.

```
Row key                 Column          Value
---------------------   -------------   ----------------------------
drive:obj:<uid>         type            'file' or 'dir'
drive:obj:<uid>         name            'filename.txt'
drive:obj:<uid>         parent          '<parent_uid>'
drive:obj:<uid>         owner           '<username>'
drive:obj:<uid>         size            '1048576'  (bytes)
drive:obj:<uid>         created         '1714000000'  (Unix epoch)
drive:obj:<uid>         modified        '1714000100'
drive:obj:<uid>         chunks          '3'  (number of chunks)
drive:obj:<uid>         mime            'application/pdf'
drive:dir:<uid>         children        'uid1,uid2,uid3,...'
drive:file:<uid>:0      data            <up to 1MB binary blob>
drive:file:<uid>:1      data            <up to 1MB binary blob>
drive:file:<uid>:N      data            <up to 1MB binary blob>
drive:root:<username>   root            '<root_dir_uid>'
```

7.2 File Chunking

Files are split into 1 MB (1,048,576 byte) chunks. Each chunk is stored as a separate KV column value. This is necessary because KV values are held in memory - a single 64 MB value would be wasteful. With 1 MB chunks, a 64 MB file uses 64 KV entries.

Why 1 MB chunks?

- KV values are byte strings in memory - no paging or streaming within a value.
- 1 MB fits comfortably in an HTTP response without chunked encoding complexity.
- Aligns well with common filesystem block sizes and memory page allocators.
- Minimises WAL entry sizes for large files (each chunk WAL entry is ~1 MB).

Chunk assembly on download

On file download, the frontend reads the 'chunks' metadata column, then reads each drive:file:<uid>:N chunk in sequence and streams them to the browser. The browser assembles the original file from the response body.

7.3 Directory Structure

Directories are KV rows with a 'children' column containing a comma-separated list of child UIDs. Adding a child updates this list using a CPUT retry loop (read current value, append new UID, CPUT old -> new). This is the standard optimistic concurrency pattern for shared append-only lists.

```
// append_child (simplified):
while (true) {
    string old_val = kv.get(dir_row, 'children');
    string new_val = old_val.empty() ? uid : old_val + ',' + uid;
    if (kv.cput(dir_row, 'children', old_val, new_val)) break;
```

```
    // retry on concurrent modification
}
```

7.4 UID Generation

Each file and directory gets a UUID-like UID generated from /dev/urandom (128 bits, hex-encoded). UIDs are globally unique across users and directories, preventing collisions even under concurrent uploads.

7.5 Move and Copy

Move: updates the 'parent' column of the object and atomically removes/adds to the parent directories' children lists. Copy: creates a new UID, copies all metadata and chunk data to new KV rows, and inserts into the destination directory.

8. Storage Quota

8.1 Quota Design

Each user has a storage quota enforced by the frontend. The default quota is 50 MB (52,428,800 bytes). The quota is checked before every upload.

```
static constexpr size_t DEFAULT_QUOTA_BYTES = 50ULL * 1024 * 1024;  // 50 MB
```

8.2 Usage Calculation

Usage is calculated by traversing the entire virtual filesystem tree of the user, summing the 'size' column of every file object. The traversal is recursive (BFS/DFS), starting from the user's root directory UID.

subtree_file_bytes algorithm

```
function subtree_file_bytes(uid, visited, depth):
    if depth > 4096: return 0   // cycle guard
    if uid in visited: return 0  // deduplication
    visited.add(uid)
    obj = kv.get_row('drive:obj:' + uid)
    if obj.type == 'file': return obj.size
    if obj.type == 'dir':
        total = 0
        for child_uid in obj.children.split(','):
            total += subtree_file_bytes(child_uid, visited, depth+1)
        return total
```

- visited set: prevents double-counting if the same UID appears in multiple directory listings (should not happen in a correct tree, but guards against bugs).
- depth cap at 4096: prevents infinite recursion if a directory cycle is ever introduced by a bug.

8.3 Quota Enforcement

Before an upload, the frontend calculates current usage + new file size. If the sum exceeds DEFAULT_QUOTA_BYTES, the upload is rejected with HTTP 413 (Payload Too Large) and an error message to the user.

```
if (current_usage + file_size > DEFAULT_QUOTA_BYTES) {
    return HttpResponse::error(413, "Storage quota exceeded (50 MB)");
}
```

8.4 Maximum File Upload Size

The maximum single file upload is bounded by the HTTP server's body limit:

```
static constexpr size_t MAX_BODY_BYTES = 64ULL * 1024 * 1024;  // 64 MB
```

This is a hard limit enforced by the HTTP parser - if the request body exceeds 64 MB, the server returns HTTP 413 immediately without reading the rest of the body.

Summary of size limits

Per-file upload limit:	64 MB - HTTP body parser hard limit (MAX_BODY_BYTES)
Per-user quota:	50 MB - Total storage across all files (DEFAULT_QUOTA_BYTES)
KV value chunk size:	1 MB - Each chunk stored as one KV column value

Max chunks per file:	64 - 64 MB / 1 MB = 64 chunks
Tablet WAL entry:	~1 MB+header - One WAL entry per chunk write

8.5 What Happens When Quota Is Exceeded?

When a user attempts to upload a file that would push them over the 50 MB quota:

- The frontend calculates subtree_file_bytes for the user's root directory.
- If current_usage + new_file_size > 50 MB: HTTP 413 is returned, no data is written.
- If the file is exactly at the boundary (e.g., user has 49.9 MB and uploads 0.2 MB): the upload is allowed (total = 50.1 MB, which exceeds quota). This is a known edge case - the quota check uses < not <= because the quota is a soft cap.
- After the upload is rejected, the user sees an error in the browser and can delete files to free space.

What happens if 64 MB HTTP limit is exceeded?

If a client sends a Content-Length header > 64 MB, the HTTP parser rejects the request with 413 before reading any body bytes. If the Content-Length is missing (streaming), the parser reads up to 64 MB and then returns 413. In both cases, no partial data is written to the KV store.

9. Email System

9.1 Mail KV Schema

```
Row key              Column           Value
------------------   --------------   ----------------------------------
<user>:mail          folder:inbox     'uid1 uid2 uid3 ...'  (space-sep)
<user>:mail          folder:sent      'uid1 uid2 ...'
<user>:mail          folder:trash     'uid1 uid2 ...'
<user>:mail          msg:<uid>        '<from>|<to>|<subject>|<date>|<size>'
<user>:mail          body:<uid>       '<full MIME body>'
<user>:mail          attach:<uid>:<n> '<base64 attachment data>'
```

9.2 Inbound Mail Flow

Inbound mail is handled by the SMTP server (port 2525). The flow:

```
External MTA  ->  SMTP Server (2525)
  EHLO, MAIL FROM:, RCPT TO:, DATA, .
SMTP Server:
  1. Validate recipient domain (@penncloud.com or @penncloud)
  2. Look up recipient in KV (check password row exists)
  3. Write message body to KV: PUT <user>:mail body:<uid> <body>
  4. Write message meta to KV: PUT <user>:mail msg:<uid> <meta>
  5. CPUT retry loop to append uid to folder:inbox list
  6. Reply 250 OK to sender
```

9.3 Outbound Mail Flow

Outbound mail from a PennCloud user:

```
Browser  ->  Frontend (POST /api/mail/send)
Frontend:
  1. Parse To:, Subject:, Body from HTTP form
  2. If recipient is @penncloud.com: call deliver_local() directly
  3. If external: open TCP to relay (port 25) or direct MX lookup
     EHLO, MAIL FROM:, RCPT TO:, DATA, body, .
  4. Write to sender's folder:sent index
```

9.4 Real-Time Inbox (SSE)

When a user opens their inbox, the frontend establishes an SSE connection. A server thread polls the user's folder:inbox KV column every 500ms. When the UID list changes (new mail arrived), it pushes a 'data: new_mail' SSE event. The browser JavaScript handles the event and refreshes the inbox view.

- SSE polling interval: 500ms (configurable).
- SSE slot limit: threads/4 (default 8 concurrent SSE connections).
- Disconnect detection: write() failure to the SSE socket closes the connection and releases the slot.

9.5 Folder Management

Users can move messages between inbox, sent, and trash. Move operations use a CPUT retry loop: remove UID from source folder list, add to destination folder list. Permanent delete removes the UID from the folder list and deletes the body and meta columns from the KV row.

10. Bottlenecks & Performance

10.1 KV Client Global Mutex

The KVClient uses a single global std::mutex per client instance to protect access to the connection pool. With 32 frontend threads all making KV calls, this mutex is a significant bottleneck. Every KV call blocks all other threads on the same frontend.

Impact

At 32 threads, throughput is bounded by 1/mutex_hold_time KV ops/sec per frontend instance. Under high load, threads queue behind the mutex.

Why not fixed?

Proper per-connection locking would require a connection pool of N connections with a semaphore. That adds significant complexity. For a course demo system, a global mutex is the acceptable trade-off.

10.2 SSE Polling (500ms KV Reads)

Each SSE connection performs a KV GET every 500ms. With 8 SSE slots, that is 16 KV reads/second just for SSE (2 per connection per second). These reads compete with user-initiated reads and writes in the connection pool.

Mitigation

SSE slot cap (threads/4) limits the total SSE load. A proper system would use KV-level push notifications (pub/sub) instead of polling.

10.3 Storage Quota Tree Walk

Calculating storage usage requires traversing the entire virtual filesystem tree. For a user with thousands of files in deeply nested directories, this is O(N) KV reads where N is the number of objects. Each KV read has a round-trip latency.

Mitigation

A production system would maintain a running usage counter updated atomically on every upload/delete. PennCloud recalculates on every upload, which is acceptable for small datasets (< 100 files) but would be slow at scale.

10.4 WAL Sequential Write

The WAL mutex serialises all writes to a single WAL file per tablet. Two concurrent writes to the same tablet cannot interleave - they queue behind wal_mu_. This limits write throughput to one write at a time per tablet.

Why acceptable?

The WAL write itself is fast (a few KB flush). The primary bottleneck is network round-trip to replicas (500ms timeout), not WAL write latency. For a course demo with a few concurrent users, WAL serialisation is never the bottleneck.

10.5 Coordinator Single Point

There is only one coordinator. If it crashes, no new primary elections can happen, and frontends cannot get

topology updates. However, cached topology means existing operations continue to work until a node failure requires an election.

10.6 Large File Upload Timeout

The KV client socket timeout is 350ms. A 64 MB file upload requires 64 sequential 1 MB KV PUT operations, each with replication forwarding. If any single PUT takes > 350ms (e.g., due to replication latency), the KV client times out and retries. This can cause false failures during large uploads under load.

11. Design Tradeoffs

Primary-backup vs. Paxos/Raft

Primary-backup is simpler to implement and debug. Paxos/Raft would give stronger consistency guarantees (no lost writes during leadership transitions) but requires significantly more code. For a course project, primary-backup is the right call.

Synchronous vs. Async replication

Synchronous replication (wait for quorum ACK before returning) gives read-your-writes consistency. Async replication would be faster but a frontend reading from a different replica immediately after a write might see stale data. Synchronous wins for correctness.

In-memory store vs. disk-backed

All data is in memory (WAL and checkpoints are durability layers, not primary storage). This gives O(1) lookup speed but limits dataset size to available RAM. A disk-backed store (like LevelDB) would support larger datasets at the cost of complexity.

Tablet-based sharding vs. consistent hashing

Fixed letter-range sharding is simple and deterministic. Consistent hashing would allow dynamic resharding as nodes are added, but is overkill for a fixed 3-node cluster.

SSE vs. WebSocket for push

SSE requires only standard HTTP - no upgrade handshake, no framing protocol, no library. WebSocket gives bidirectional communication (not needed here) at the cost of a more complex protocol. SSE is the right fit for server-to-client notifications.

Global KV client mutex vs. per-connection pool

A global mutex is simple but serialises all KV calls on a frontend. A full connection pool (N connections, semaphore) would allow parallel KV calls at the cost of more complex lifecycle management (connection death, pool drain on shutdown).

50 MB quota vs. per-user configurable

Fixed quota simplifies the admin model. Configurable per-user quota would require an admin API and a quota row in KV per user. For a demo, fixed is fine.

ps+awk kill vs. lsof/fuser/ss

lsof/fuser/ss are not available in Docker containers. ps+awk matching on the --port argument is universally available and achieves the same result.

1 MB chunk size vs. variable

Fixed chunk size simplifies chunked retrieval (chunk index = byte_offset / CHUNK_SIZE). Variable chunk size would allow better space efficiency for small files but adds complexity in the metadata schema.

24-hour session TTL vs. sliding window

A fixed 24-hour TTL is simple to implement (one timestamp comparison). A sliding window (TTL resets on every request) would require a KV write on every authenticated request, adding latency. Fixed TTL is the right trade-off for a course demo.

12. KV Wire Protocol

The frontend communicates with KV nodes over a custom binary-framed text protocol over TCP. Commands are newline-terminated text; binary payloads (values) are length-prefixed and follow the command line.

12.1 Request Format

```
GET <row_len> <col_len>\r\n<row><col>
PUT <row_len> <col_len> <val_len>\r\n<row><col><val>
CPUT <row_len> <col_len> <old_len> <new_len>\r\n<row><col><old><new>
DELETE <row_len> <col_len>\r\n<row><col>
```

12.2 Response Format

```
+OK\r\n                          (success, no value)
+OK <val_len>\r\n<val>           (success with value)
-ERR <message>\r\n              (error)
-NOT_FOUND\r\n                  (GET on non-existent key)
-FAIL\r\n                       (CPUT old value mismatch)
```

12.3 Replication Protocol

```
Primary  ->  Replica:
REPLICATE <tablet> <lsn> PUT <row_len> <col_len> <val_len>\r\n<row><col><val>
REPLICATE <tablet> <lsn> DELETE <row_len> <col_len>\r\n<row><col>

Replica  ->  Primary:
+OK LSN=<lsn>\r\n
```

12.4 Coordinator Protocol

```
KV Node  ->  Coordinator:  REGISTER <id> <host> <port> <tablet>\r\n
Coordinator  ->  KV Node:  +OK\r\n

Coordinator  ->  KV Node:  PING\r\n
KV Node  ->  Coordinator:  +PONG\r\n

Coordinator  ->  KV Node:  PROMOTE\r\n       (become primary)
Coordinator  ->  KV Node:  DEMOTE\r\n        (become secondary)
Coordinator  ->  KV Node:  ADD_REPLICA <id> <host> <port>\r\n

KV Node  ->  KV Node:     SYNC_FROM <tablet> <ckpt_ver> <lsn>\r\n
Primary  ->  Replica:     +SNAPSHOT <len>\r\n<blob>
                       or +DELTA <len>\r\n<blob>
```

13. Startup & Configuration

13.1 Process Inventory

node1: Port(s): 5000 (KV), 5100 (repl) - KV store node 1, tablets A/B/C primary

node2: Port(s): 5001 (KV), 5101 (repl) - KV store node 2, tablets A/B/C backup

node3: Port(s): 5002 (KV), 5102 (repl) - KV store node 3, tablets A/B/C backup

coordinator: Port(s): 6000 - Heartbeat, election, topology

fe1: Port(s): 8080 - Frontend HTTP server 1

fe2: Port(s): 8090 - Frontend HTTP server 2

fe3: Port(s): 8095 - Frontend HTTP server 3

smtp_server: Port(s): 2525 - SMTP inbound/outbound

13.2 Frontend CLI Flags

```
--port        <N>       HTTP listen port (default 8080)
--kv-host     <host>    KV node host (default 127.0.0.1)
--kv-port     <N>       KV node port (default 5000)
--threads     <N>       thread pool size (default 32)
--id          <str>     server identity label (default fe1)
--static      <dir>     static files directory (default ./static)
--coord-host  <host>    coordinator host (default 127.0.0.1)
--coord-port  <N>       coordinator port (default 6000)
--redirect-to <url>     run as redirect stub to this URL
--page-not-found-stub   run as 404 stub (all requests -> 404)
```

13.3 Startup Script

start_multi_tablet_cluster.sh launches all 8 processes in order: KV nodes first (with data directories), then coordinator, then frontends, then SMTP. Each process is backgrounded with & and the PID is recorded for shutdown.

14. Security Considerations

14.1 Implemented Mitigations

HttpOnly session cookie:	Session ID not accessible to JavaScript - prevents XSS session theft.
128-bit random SID:	Brute-force resistant session ID (2^128 entropy from /dev/urandom).
CPUT for atomic updates:	Prevents lost-update race conditions on shared KV values.
JSON control-char escaping:	JSON strings sanitise 0x00-0x1F to prevent JSON injection.
Admin loopback check:	Admin control endpoints reject requests not originating from 127.0.0.1.
SMTP domain validation:	SMTP server only accepts mail for @penncloud.com / @penncloud.

14.2 Known Weaknesses (Out of Scope for Course)

No TLS:	All HTTP and KV traffic is plaintext TCP. Credentials are sent unencrypted.
No CSRF protection:	State-changing API endpoints lack CSRF tokens.
No rate limiting:	Login endpoint has no brute-force protection.
Content-Disposition injection:	Filenames in download headers are not fully sanitised.
Open redirect:	Admin redirect URL comes from request Host header (not validated).
SMTP dot-stuffing:	Only first leading dot removed; RFC 5321 violation for multi-dot lines.

15. Summary

PennCloud is a functionally complete distributed cloud platform built entirely in C++17 from scratch. It demonstrates the full stack of distributed systems concepts:

- Replicated storage with WAL, checkpointing, and delta/snapshot resync.
- Primary election driven by highest-LSN selection after ~1.5s failure detection.
- Stateless frontend tier with KV-backed sessions and SSE push notifications.
- Virtual filesystem with 1 MB chunked storage, CPUT-based directory updates, and per-user 50 MB quota enforcement.
- SMTP inbound/outbound email with MIME attachment support.
- Admin console with kill/restart/redirect-stub for live frontend HA demonstration.

Key Numbers

Max file upload:	64 MB (HTTP body limit)
Per-user quota:	50 MB (soft, checked pre-upload)
File chunk size:	1 MB per KV value
Session TTL:	24 hours
Session ID entropy:	128 bits (/dev/urandom)
Failure detection:	~1.5 seconds (3 × 500ms heartbeats)
Replication quorum:	2 of 3 nodes (majority)
SSE poll interval:	500ms
KV socket timeout:	350ms
KV retry backoff:	0 / 75 / 150 / 250ms
Frontend threads:	32 per server
SSE slot limit:	8 (threads/4)